

0.10.3. La fonction `textS()` — un texte qui passe à la ligne

`#include <textS>` : comme `texte()`, mais arrivé au bord droit, la suite repart en colonne 0 de la ligne d'en dessous.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `textS(colonne, ligne, "TEXTE")` écrit comme `texte()`, mais **ne s'arrête pas au bord** : après la colonne 19, la lettre suivante va en colonne 0 de la ligne d'en dessous. Après la ligne 17, elle remonte en haut.

La ligne à écrire : `#include <textS>`.

Ses arguments : les mêmes que `texte()` : colonne, ligne, texte entre guillemets.

Ce qu'elle coûte : environ **250 octets** la première fois : elle calcule la case de chaque lettre pendant le jeu.

Essaie : `textS(0, 0, "...")` avec un texte de 45 lettres : il remplit deux lignes et demie.

On retrouve `textS` dans 3 leçons

[0.11](#), [0.11.1](#), [0.11.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **336 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         29 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        96 o ← CopierTuiles – 6 lettres sur 43
"BONJO"        5 o ← texte() – un texte gravé
"UR"           2 o ← texte() – un texte gravé
```

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
texte() × 2    22 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
textS() × 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

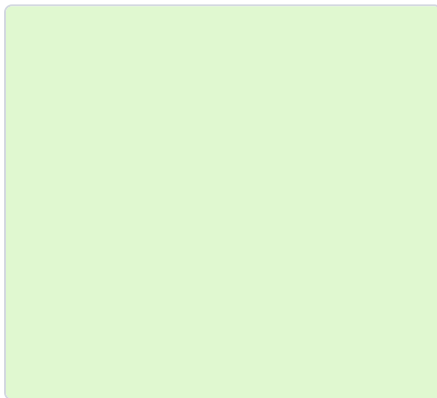
```
// ---- #include <textS> : un texte qui passe à la ligne ----  
// BONJOUR commence en colonne 15 : B O N J O tiennent jusqu'à la colonne 19,  
// puis U et R repartent en colonne 0, une ligne plus bas.  
  
#include <textS> // textS() : le texte qui va à la ligne tout seul
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    textS(15, 8, "BONJOUR"); // BONJO en (15..19, 8), puis UR en (0..1, 9)  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

BONJO tout à droite de la ligne 8, et UR au début de la ligne 9.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ BONJO tient jusqu'au bord droit
- ✓ UR repart en colonne 0, une ligne plus bas (« UR »)